

Airframe Tuning Normalization — From Observed Data to FC Limits

Purpose: Establish a defensible, data-driven basis for the FC parameter (`lo`, `hi`) bounds. The current FC (`lo`, `hi`) in `params.c` were authored by hand/“educated guess”; this document tests the alternative — derive the global limits from the observed tuning ranges across all shipped `.af` airframes.

Method: For every parameter, collect the raw FC-native values actually tuned into each airframe, take the **min of the mins** and the **max of the maxes**, and expand by a margin factor ($\times 2$, $\times 0.5$) to give headroom. Compare against the current class-based FC limits.

1. Why this matters / how limits are currently set

- FC is the **source of truth** for hard clamps (`ParamClass[]` + `ParamTable` in `UAVXArmQ/src/params.c`). The FC does **not** clamp incoming writes further — the `ParamTable` max *is* the only agreement between FC and GCS.
- GCS `parameters.py` mirrors those bounds exactly; `test_param_limits.py` refuses to pass if they drift.
- Every `.af` file also carries a `[LIMITS]` block giving a **per-airframe tuning sub-range** inside (or clamped to) the class ceiling. Those sub-ranges were generated by `migrate_limits.py` as `[v*0.5, v*2.0]` — i.e. $\pm 2\times$ around the shipped value, not a true observed min/max.

So today, both the FC hard bounds *and* the `.af` sub-ranges are somewhat arbitrary. The cross-airframe scan below gives an independent sanity check.

2. Data gathered

18 airframes scanned (`generic/*.af` + `user/*.af`):

category	files
MR (quads/hex/oct)	Quad, Quad_Medium, Quad_Racer, Hex, Oct, RudderElevator, Arado_555, Arado_555_Tuned
FW (fixed-wing)	SkySurfer_Bixler, Radian, Radian_Tuned, Dragon, Delta, Elevon, Spoileron, SmallSpoileron, Shadow, Horten

Observed values are **raw FC-native units** (radians for angles, fractions for percent, not degrees/%).

3. Observed ranges vs current FC limits (FLOAT tuning params)

Observed = actual min/max shipped across all frames. Proposed = observed $\pm 2\times$ margin (min-of-mins $\times 0.5$ floored at 0, max-of-maxes $\times 2$).

tag	param	obs (min,max)	FC lo	FC hi	proposed +margin
0	RollRateKp	0, 0.91	0	3	0, 1.83

tag	param	obs (min,max)	FC lo	FC hi	proposed +margin
2	RollAngleQKp	0.52, 11.2	0	20	0, 22.4
5	PitchRateKp	0, 2.5	0	3	0, 5
7	PitchAngleQKp	1, 10	0	20	0, 20
10	YawRateKp	0, 0.7	0	3	0, 1.4
11	RollRateKd	0, 0.137	0	0.2	0, 0.27
23	RollAngleQKi	0, 13.5	0	25	0, 27
24	PitchAngleQKi	0, 3	0	25	0, 6
27	PitchRateKd	0, 0.015	0	0.2	0, 0.03
28	NavVelKp	0, 2.4	0	15	0, 4.8
29	AltVelKd	0, 0.026	0	3.66	0, 0.052
40	NavPosIntLimit	5, 12	0	15	0, 24
48	NavCrossTrackKp	0, 0.04	0	15	0, 0.08
56	NavPosKp	0, 1.5	0	15	0, 3
60	NavPosKi	0, 0.005	0	15	0, 0.01
63	MaxYawRate	1, 10.47	0	10.47	0, 20.9
82	MaxRollRate	0.785, 4.19	0	10.47	0, 8.38
83	MaxPitchRate	0.785, 3.93	0	10.47	0, 7.85
88	MaxHeadingRate	0.524, 1.05	0	10.47	0, 2.09
67	FWMaxClimbAngle	0.524, 1.047	0	1.047	0, 2.09
68	NavMaxAngle	0.349, 0.698	0	1.047	0, 1.40
74	MaxPitchAngle	0.524, 1.047	0	1.047	0, 2.09
76	MaxRollAngle	0.524, 1.047	0	1.047	0, 2.09
78	NavHeadingTurnout	0.349, 0.7	0	1.047	0, 1.40
64	FWRollPitchFF	0, 0.3	0	1	0, 0.6
65	FWPitchThrottleFF	0, 0.01	0	1	0, 0.02
70	FWAileronDifferential	0, 0.3	0	1	0, 0.6
86	FWAileronRudderMix	0, 1	0	1	0, 2
87	FWAltSpoilerFF	0, 0.5	0	1	0, 1
114	AHThrottleMovingTrigger	0.02, 0.2	0	1	0, 0.4
19	EstCruiseThr	0.2, 0.55	0	1	0, 1.1
20	StickHysteresis	0.02, 0.02	0	1	0, 0.04
21	FWClimbThrottle	0, 0.7	0	1	0, 1.4
22	PercentIdleThr	0, 0.05	0	1	0, 0.1
39	RollCamTrim	0, 0.01	0	1	0, 0.02

Observations

1. **Most current FC ceilings are far wider than observed use** — e.g. NavVelKp FC hi=15 vs observed 2.4; NavPosKp 15 vs 1.5; AltVelKd 3.66 vs 0.026; FWPitchThrottleFF 1 vs 0.01. The hand-authored bounds are overly generous by roughly 3-100×. Wide bounds are *safe* but lose quantization resolution and make the value harder to read (a 0.01 range shown as 0..15).
2. **No observed value falls outside the FC range** — the three “OUT OF RANGE” flags (tags 67/74/76) were float-format artifacts ($1.0472 \approx 1.047198$), not real violations. Good sign: the current bounds genuinely contain the tuned fleet.
3. **The ×2 margin from the observed max-of-maxes is already narrower than the current FC hi** for every row above except tags 2/23/96 (angle gains), which are already near their ceilings.

Params that should NOT follow this scheme

- **Enums / selectors** (U8: ArmingMode, RxType, AFTType, ConfigBits, ...) — bounds are discrete index ranges, not tunable.
- **Physical/hard limits** (LowVoltThres, Voltage/Current scale, BatteryCapacity, NavFenceRadius, NavProx, VRSROC, DiveRecoverAlt) — bounds come from the sensor or safety envelope, not from tuning spread.
- **Fixed structural params** (ServoLPFHz, YawLPFHz, AccConfSD, Insight/KF variances, StickHysteresis) — set once, not tuned per craft.

4. Physics → gain relationship

The seed for tuning already exists in `GCS_get_scale_factors()` (`parameter_window.py:2431`). It scales a reference aircraft's base curve by phys-derived ratios. These are the **same relationships** you'd use to derive sane per-craft limits from the physical description:

MR (reference: 800 g, 220 mm arm)

- **Rate Kp/Kd** $\propto 1 / \text{inertia_ratio}$, where $\text{inertia_ratio} = (\text{arm}/\text{ref_arm})^2 \cdot (\text{mass}/\text{ref_mass})$ → lighter / shorter-arm quads need *lower* rate gains.
- **Angle Kp/Ki / IntLimit** $\propto \text{mass_ratio}$ → heavier → more authority.
- **Max rate** $\propto 1 / \text{mass_ratio}$ → lighter → faster response possible.

FW (reference: 1000 g, 1800 mm wing)

- **Rate Kp/Kd** $\propto 1 / \text{inertia_ratio} = 1 / ((\text{span}/\text{ref_span})^2 \cdot \text{mass_ratio})$ → larger wingspan → more inertia → higher gains.
- **Angle Kp/Ki / IntLimit** $\propto \text{mass_ratio}$.
- **Max rate** $\propto 1 / \text{mass_ratio}$.
- Nav terms additionally $\propto 1 / \text{cruise_speed}$.

Implication: The FC limits don't need to be per-phys — they are *hard ceilings*, and the GCS already projects a per-craft point inside them via the phys descriptor + character slider. But the **recommended range** a user is offered ([LIMITS] sub-range) *should* follow these ratios so the slider is not spent mostly over impossible values for a given frame.

Implemented: per-craft usable bounds from phys

`migrate_limits.py` now generates the [LIMITS] sub-range with phys-relative scaling, mirroring the GCS tuner's `_get_scale_factors()` ratios:

```
rate_kp_range   = v * sf * [0.5, 2.0],   sf = 1/inertia_ratio   (MR + FW)
angle_kp_range  = v * sf * [0.5, 2.0],   sf = mass_ratio
max_rate_range  = v * sf * [0.5, 2.0],   sf = 1/mass_ratio
```

where `v` is the stored value, $\text{inertia_ratio} = (\text{arm}/\text{ref_arm})^2 \cdot \text{mass_ratio}$ (MR, ref 800 g / 220 mm) or $(\text{span}/\text{ref_span})^2 \cdot \text{mass_ratio}$ (FW, ref 1000 g / 1800 mm). The reference craft maps to `sf = 1`, leaving its sub-range unchanged. This ties the slider's span to the aircraft's actual inertia/mass, so the bar maps onto physically reachable gains.

All 18 shipped `.af` files were regenerated with the phys-relative limits (e.g. Quad_Medium ROLL_RATE_KP range 0.0383-0.153, Quad_Racer 0.242-0.968) and revalidated by `test_param_limits.py`.

5. Recommendation

1. **Do not shrink the FC hard bounds** just because observed use is narrow. Wide ceilings are safe, and quantizing a 0.01-scale value over 0..15 only affects the *spinbox* resolution (GCS steps = range/50), not flight physics.
2. **Optionally** tighten the very widest FC ceilings (NavVelKp 15→6, AltVelKd 3.66→0.5, FW-PitchThrottleFF 1→0.2) to improve read/write resolution — but this is cosmetic and must be mirrored into `parameters.py` + `test_param_limits.py` in lockstep. Deferred.
3. **Adopt the phys-relative [LIMITS] sub-range (§4) — done.** The tuner slider for each airframe now spans physically relevant values. This was the real value of the relationship you suspected.
4. Keep enums / physical / safety params off this scheme.

6. Open question

FC hard ceilings left as-is (option (a)). Rewriting the FC ParamTable hi bounds (option (b)) is lower value for higher risk and is not planned.